

CI/CD пайплайны

Правдин Иван

24.05.2025

Наш webhook сервер работает:

- Git push → автоматический деплой
- Разные ветки обновляют разные окружения

Но:

- Изменения вебхук сервера требуют ручного вмешательства.
- Нет простого доступа к логам скриптов, нет статуса деплоя.

В реальных проектах используют готовые платформы:

- **Jenkins** — самый популярный, self hosted
- **GitHub Actions** — встроен в GitHub, есть бесплатный режим
- **GitLab CI** — встроен в GitLab, есть self hosted режим
- **Azure DevOps, TeamCity, CircleCI...**

Общая идея: вместо своего webhook сервера используем платформу (отдельное приложение), которая позволяет выполнять скрипты по событиям в Git репозитории.

Плюсы:

- Надежность
- Масштабируемость
- Удобство
- Интеграции
- Переиспользуемые скрипты автоматизации

Минусы:

- Дополнительная сложность
- Свой язык автоматизации
- Ограниченные возможности

У всех этих платформ есть общая концепция — **Pipeline**

Pipeline — структурированная последовательность шагов автоматизации

Типичные шаги:

1. Получить код из Git
2. Собрать приложение (если нужно)
3. Запустить тесты
4. Задеплоить на сервер

Большинство типовых шагов уже написаны для всех популярных платформ.

Как GitHub Actions реализует концепцию Pipeline:

Базовые понятия:

- **Workflow** — описание pipeline в YAML файле
- **Job** — группа связанных шагов (например, "тестирование")
- **Step** — отдельная команда или действие
- **Action** — готовый переиспользуемый компонент

Структура: Workflow содержит Jobs, Job содержит Steps

Триггеры: push, pull request, расписание, ручной запуск

Задача: воспроизвести логику нашего webhook сервера в GitHub

Что покажем:

- Как добавить workflow в GitHub репозиторий
- Описание workflow
- Работа с секретами в GitHub Actions

Демо-репозиторий:

<https://github.com/prafdin/devops-demo-website/tree/cicd-demo>

Наш pipeline — это CI/CD pipeline

Поздравляем! Мы только что создали CI/CD pipeline

CI/CD — собирательный термин для автоматизации задач разработки:

- **CI** — Continuous Integration (непрерывная интеграция)
- **CD** — Continuous Deployment (непрерывное развертывание)

В нашем случае:

- **CI** — запуск тестов `test.sh`
- **CD** — обновление файлов сайта `deploy.sh`

CI задачи это шлюзы:

- Линтинг
- Тесты
- Сканеры безопасности

Цель: убедиться что код рабочий

CD задачи это преобразования:

- Сборка приложения
- Публикация и обновление приложения

Цель: изменить состояние артефакта

CI + CD = Continuous Delivery (непрерывная доставка)

- Pipeline — это эволюция нашего webhook сервера
- CI/CD платформы решают проблемы самодельных решений
- CI - проверки, CD - преобразования

Вопросы?